

JITServe: SLO-aware LLM Serving with Imprecise Request Information

UIUC, Google, Cisco Research

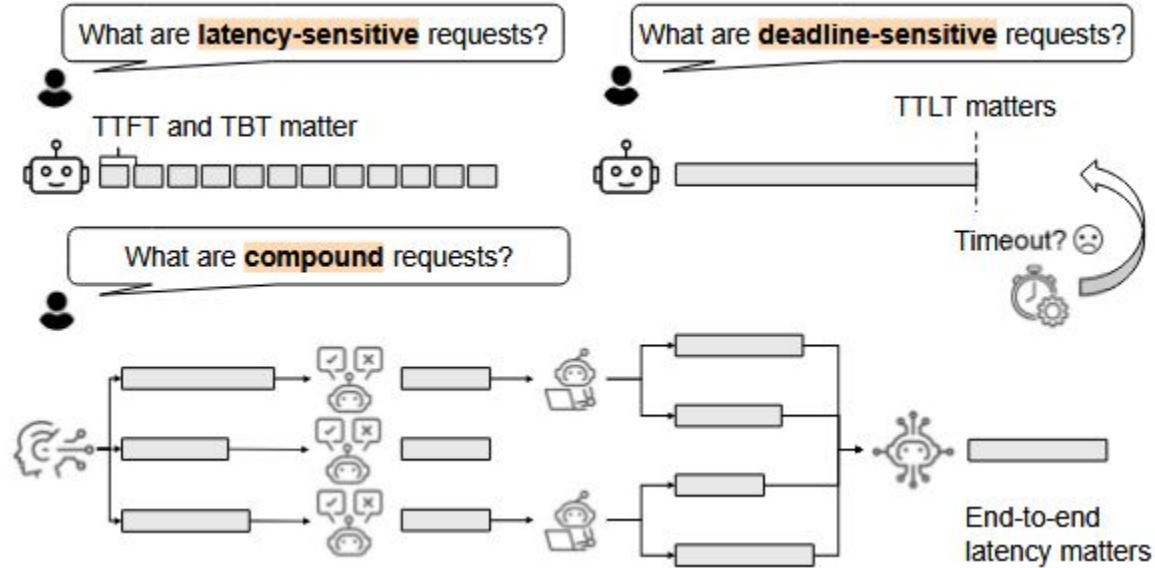
Presented by Nikhita Punati, Otto PIRamuthu, Shreenija Daggavolu

Motivation: LLM Serving is varied



- LLM apps range from chatbots to multi-agent systems
- These have different types of **service-level objectives** (SLOs) for latency
- This work introduces a scheduler that can handle these varied requests

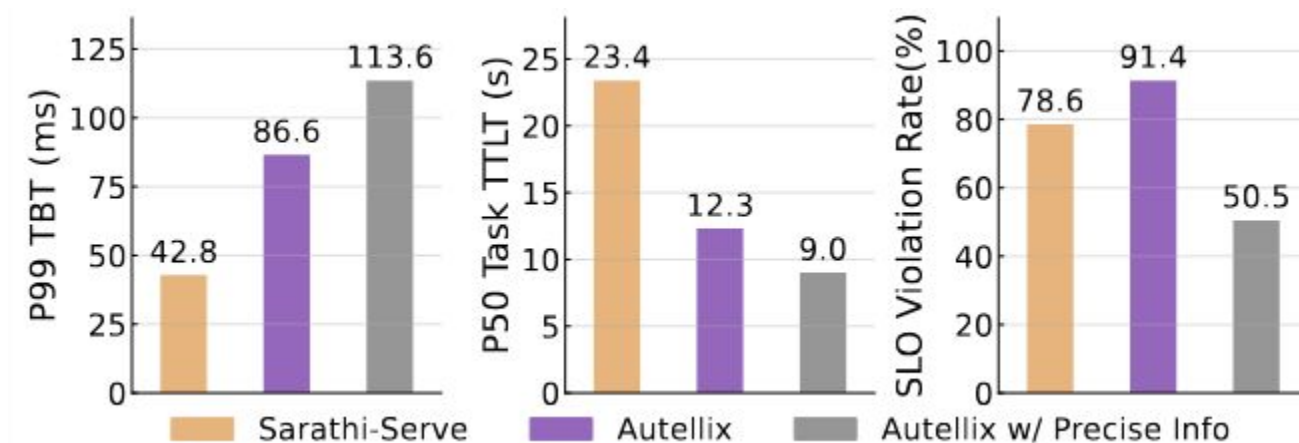
Types of Requests



Limitations of Existing Systems



- Existing systems optimize mean latency with SJF and LAS, or throughput
- But with high SLO violation rates (>90%)
- Faster-than-needed responses waste resources

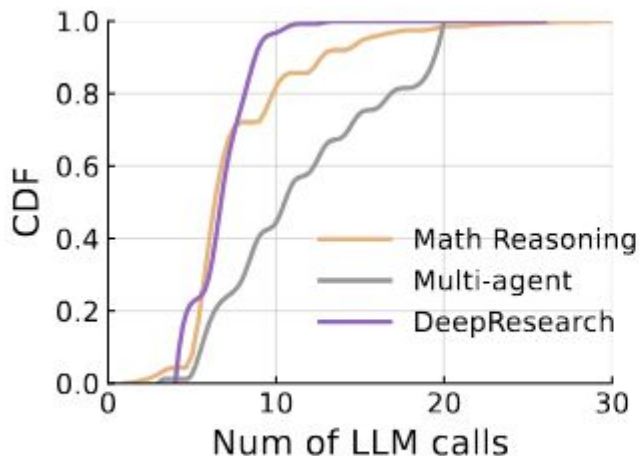


Objective: Service Goodput

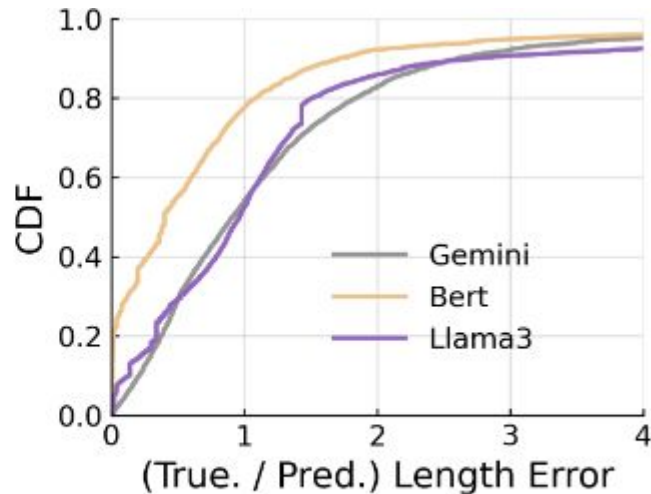


- Defined as the total number of tokens that meet request SLOs
- Requests that miss SLOs have zero value and captures wasted capacity from faster responses
- Shifts from throughput to useful throughput

- Unknown output length and execution dependencies
- Mixed request types and SLOs
- Batching constraints



(a) Varying number of LLM calls.



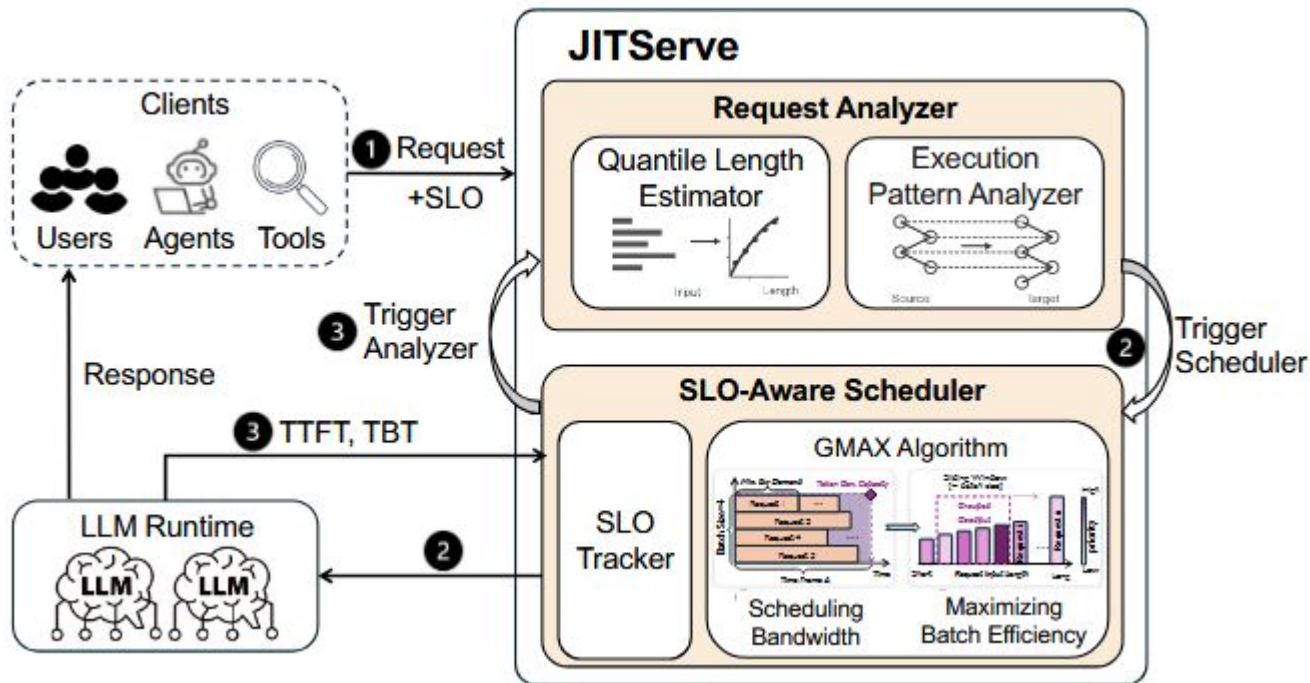
(b) Large prediction deviations.

Goal: Maximize goodput, not just speed!

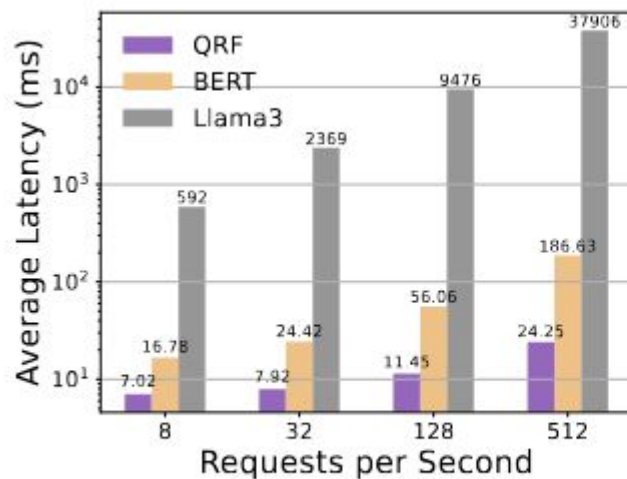
- Allocate just enough compute to meet SLO
- Start conservatively and free unused capacity for other requests

Goodput: the amount of work that actually meets SLOs so anything that misses deadlines has zero value

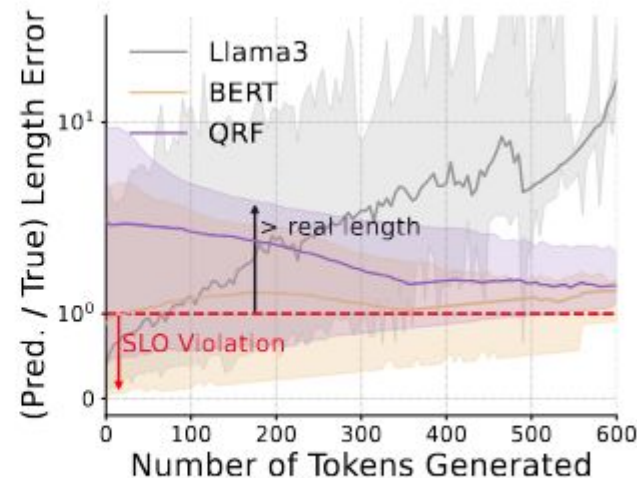
JITServe System Overview



- Use Quantile Regression Forest (QRF) to predict an upper bound on output length
- Periodically refine during generation



(a) Estimation overhead.

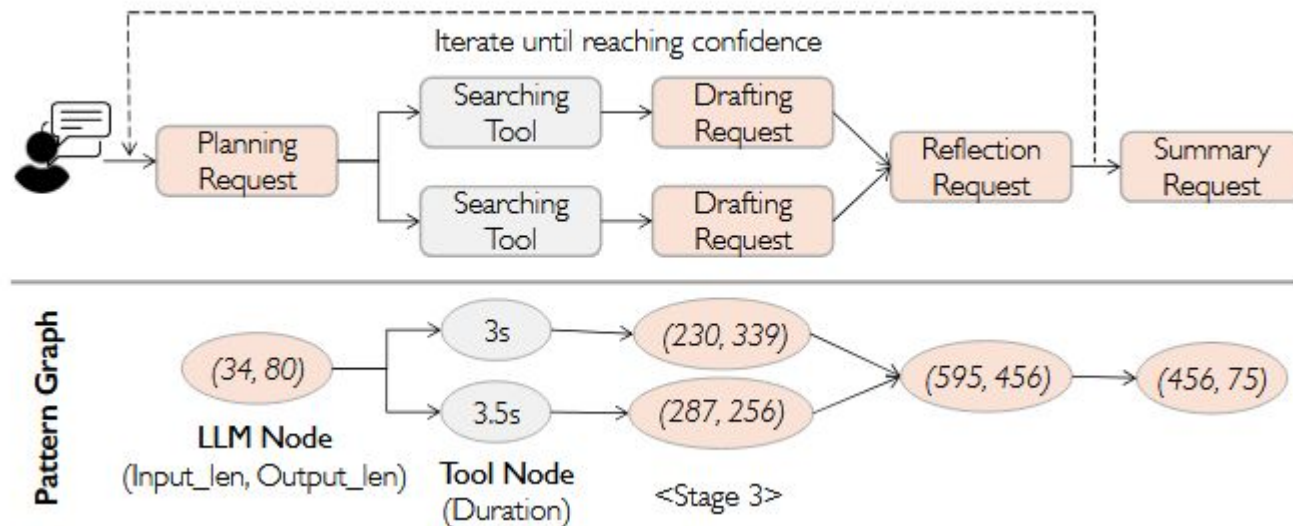


(b) Estimation accuracy.

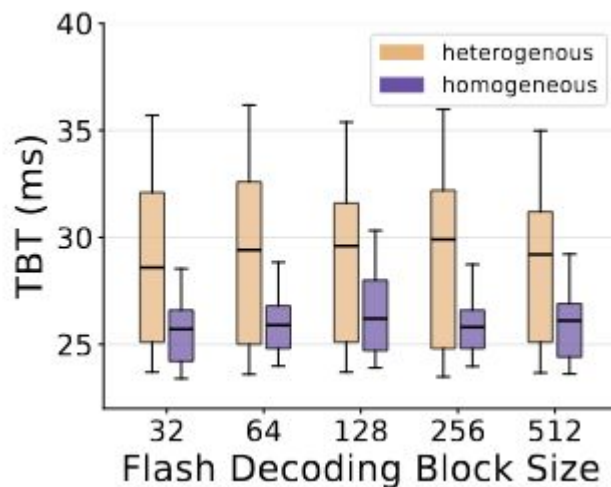
Execution Dependencies



- Represent requests as execution graphs
- Match against historical patterns
- Assign deadlines for every stage

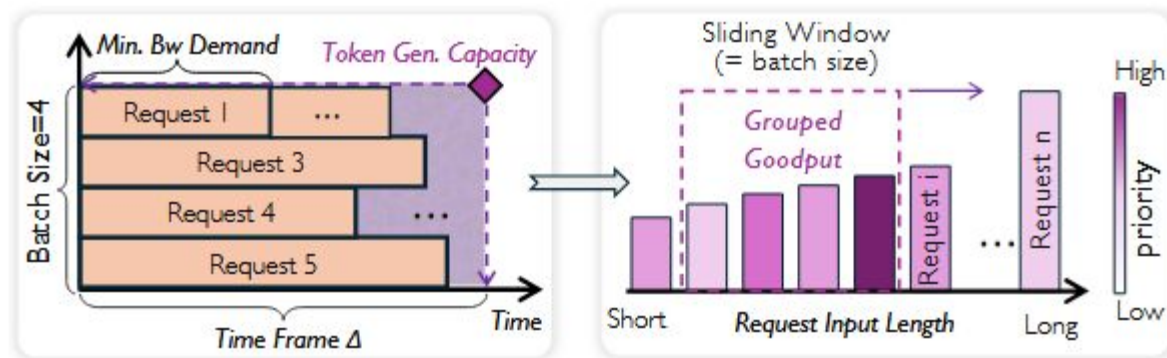


- Which requests should be served?
- How should they be batched?
- Heterogeneous lengths slows down execution



- GMAX prioritizes value density, not just speed ... unlike Shortest-Job First (SJF) and Earliest-Deadline First (EDF)
- For each request:
 - Estimate remaining work
 - Estimate time to deadline
- Compute **priority**: goodput / compute cost
 - High value + low cost → HIGH priority

- Filter high-priority requests
- Sort by input length
- Select best group (max total priority) by sliding window

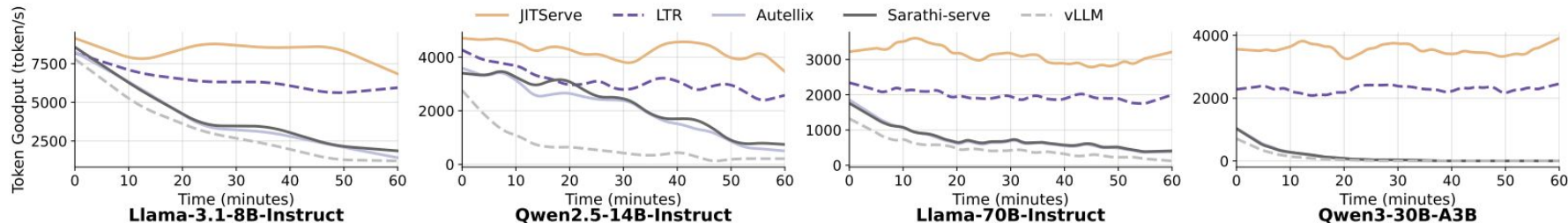


- vLLM & Sarathi
 - Optimize token-level latency (TTFT/TBT)
 - Not SLO-aware (ignore deadlines)
- Autellix & SJF
 - Optimize mean E2EL
 - Not SLO-aware (ignore SLO diversity)
- JITServe
 - SLO-aware scheduling
 - Handles heterogeneous SLOs (latency, deadlines, compound)
 - Optimizes service goodput

- Built on vLLM
- ~2800 lines of code
- Uses:
 - gRPC for prediction
 - OpenAI-style API

Workload	Req Type	Metric	Mean	Std.	P50	P95
Chatbot	Single	Input	93	244	27	391
		Output	318	313	225	1024
	Compound	Input	1300	912	1097	2767
		Output	4458	1176	4417	6452
Deep Research	Single	Input	1911	2781	403	7573
		Output	534	644	410	1544
	Compound	Input	12223	8407	10807	29282
		Output	3541	2370	3148	7525

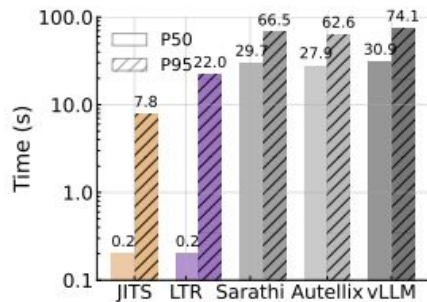
Table 2: Our evaluations include four popular applications: Chatbot, Deep Research, Agentic CodeGen, and Math Reasoning. This table shows example request length statistics for two of them.



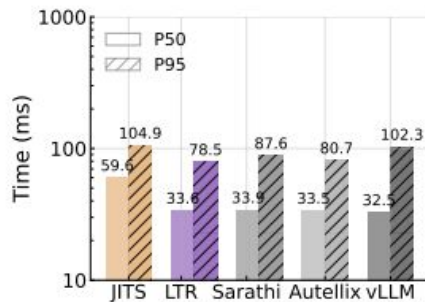
JITServe consistently maintains higher goodput over time, while baselines degrade due to SLO violations

Outperforms LTR by **1.3x - 1.7x** and Autellix by **5.3x - 6.1x** in goodput

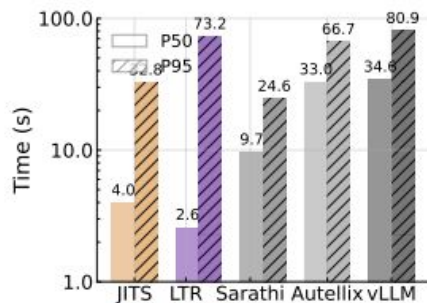
28.5%–83.2% resource savings



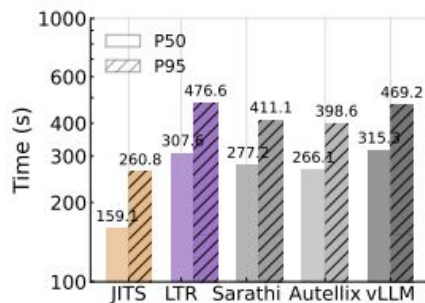
(a) Latency-sensitive TTFT (s)



(b) Latency-sensitive TBT (ms)



(c) Deadline-sensitive E2EL (s)



(d) Compound E2EL (s)

JITServe performs well across all request types, not just one specific workload

Challenge	JITServe's Idea	Result
Diverse SLOs (chat, API, agents)	SLO-aware scheduling (optimize goodput)	More requests meet deadlines
Unknown output length	Predict upper bound + refine during execution	Fewer SLO violations
Unknown dependencies (multi-step workflows)	Pattern graph matching for stage-aware scheduling	Better end-to-end latency control
Inefficient scheduling & batching	GMAX (goodput/cost) + length-aware batching	Higher goodput and GPU efficiency

- Handles diverse workloads
- Robust to uncertain inputs
- Improves application-level performance
- Easy integration with existing code

- Effectiveness depends on prediction quality
 - If QRF predictions are wrong → scheduling breaks
 - Real LLM outputs are very noisy
- Pattern graph idea (new workloads)
- Requires meaningful SLO definitions

Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot

Moonshot AI + Tsinghua University

The Case for Disaggregation

The Workload Problem

Long-context windows
dominate request traffic

Prefill is compute bound

Prefill and decoding
interfere

Strict SLOs

Cannot violate
Time-to-first-Token (TFFT)
+ Time-Between-Tokens
(TBT) SLOs

GPU resources are limited
+ inelastic

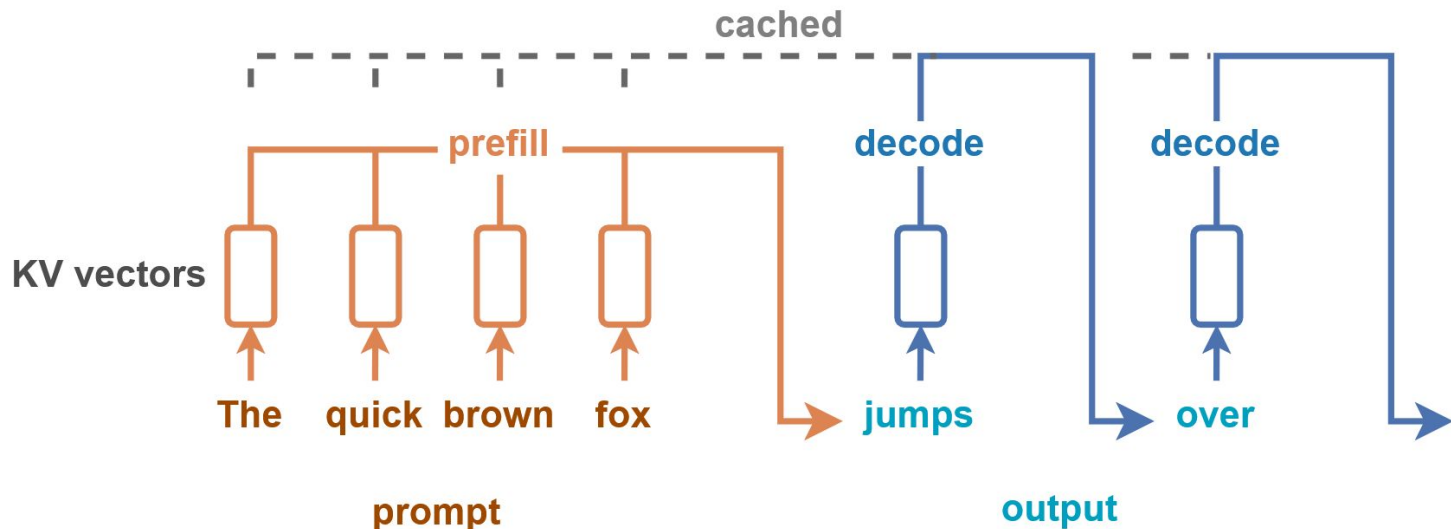
Resource Imbalance

GPU compute is utilized

GPU VRAM, DRAM, SSD,
NIC (other cluster
resources) are
underutilized!

**Let's trade cheap storage for expensive compute
by disaggregating cluster resources**

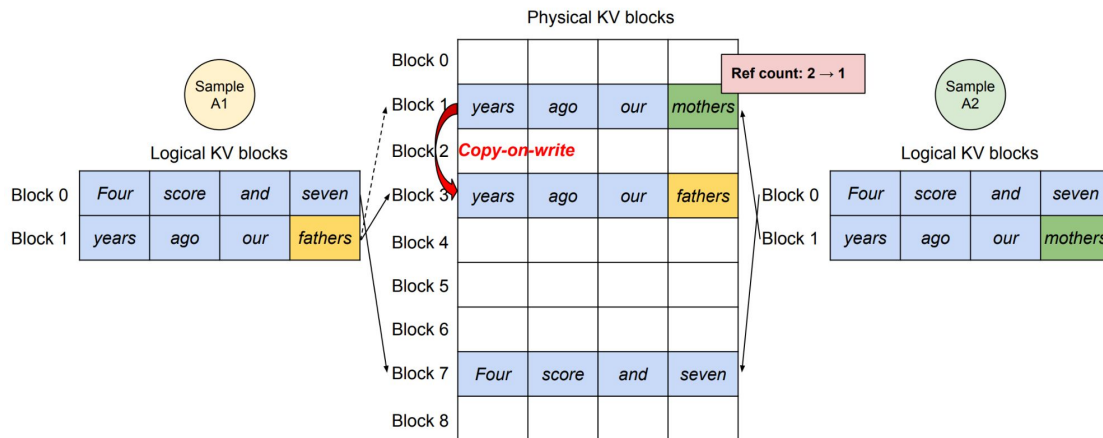
Prefilling vs Decode Stages



- **Computes KV vectors for input tokens**
 - **Highly parallelizable**
 - **Compute-bound**
- **Generates output tokens**
 - **Autoregressive**
 - **Sequential**
 - **Memory-bound**

The KV Cache + Prefix Reuse

- Decode depends on KV vectors of all previous tokens
- KV-Cache: avoid recomputation of KV vectors on each new token generation
- Append token KV to cache after computation



From Paged Attention

What are the strict SLOs?

1

Time-to-first-Token (TTFT)

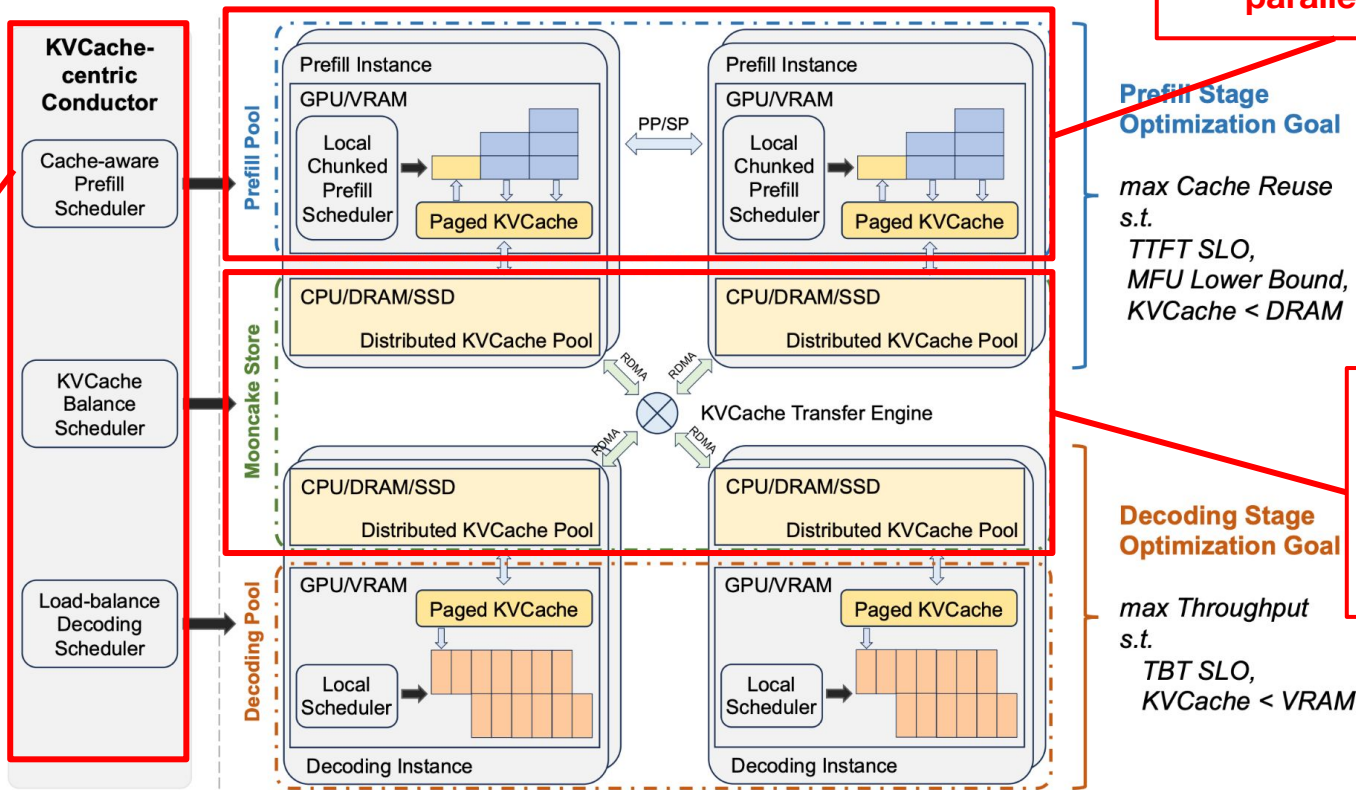
time between request arrival and generation of the first output token
Used to evaluate the **prefill** stage

2

Time-Between-Tokens (TBT)

Latency between successive token generation in a single request
Used to evaluate **decode** stage

Overview of the Mooncake Architecture



Cache-aware Conductor:
hot block replication,
P+D balancing

Prefill Pool with chunked pipeline parallelism

Prefill Stage Optimization Goal

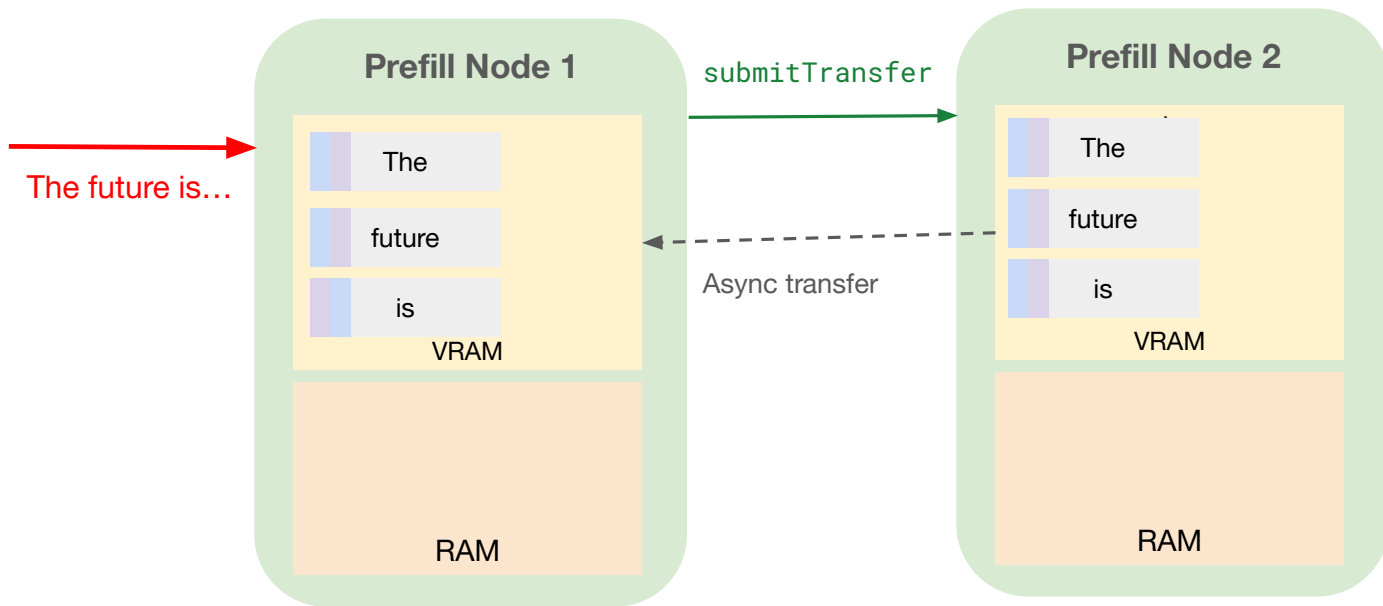
$\max$ Cache Reuse
s.t.
TTFT SLO,
MFU Lower Bound,
KVCache < DRAM

Mooncake Store:
distributed KV cache pool

Decoding Stage Optimization Goal

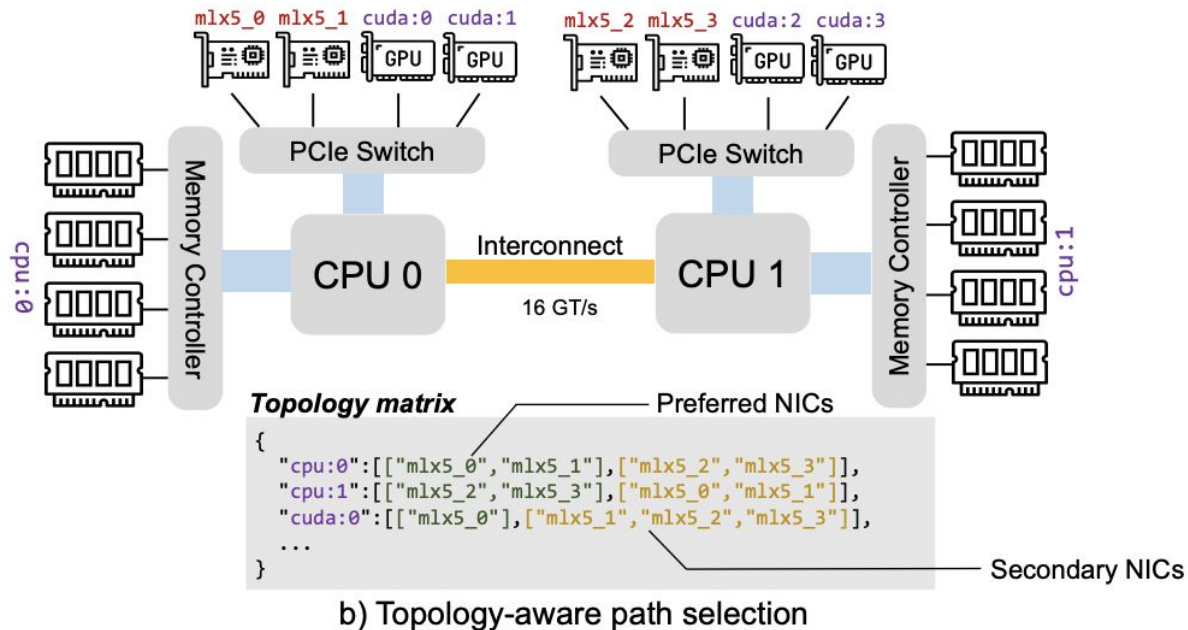
$\max$ Throughput
s.t.
TBT SLO,
KVCache < VRAM

Mooncake Store: KV-Cache



- Each block has address + hash metadata
 - Block size is between 16 - 512 tokens
 - LRU for eviction
 - Hot blocks are replicated

Mooncake Store: Transfer Engine



- Topology-aware path selection
- Distributed transfers (via chunks) across RDMA NICs simultaneously
- Endpoint pooling to prevent excessive connections
- Failure Handling for unresponsive NICs.

Ex: transfer from buffer 1 on CPU1 to buffer 0 at CPU 0

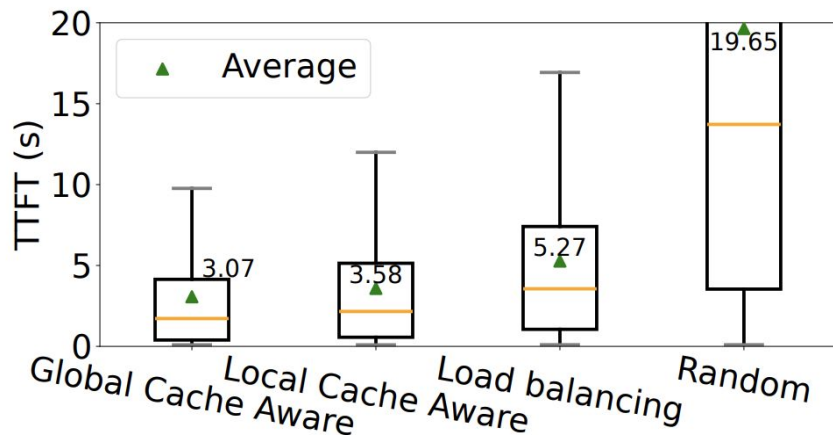
Prefill Pool

- **Motivation:** Chunked Prefill is insufficient
- The Issue with Long Contexts
 - Single GPU Prefill Bloats TTFT!
- Tensor Parallelism?
 - All-reduce operations per layer!
- Sequence Parallelism?
 - Cross-node transfers compete with KV cache transfers
- **Solution: Chunked Pipeline Parallelism (CPP)**
 - Split input into chunks,
 - Each chunk's prefill on different node
 - Cross-node communication on pipeline boundaries only
 - Works with short + long context lengths

KV-Cache Aware Conductor + Scheduling

Prefill Global Scheduling

- Route to the node with the best prefix match, not just the lightest load
- If no instance can meet the SLO, reject early



Cache Load Balancing

- Standard load balancing ignores cache state
- Hot blocks naturally replicate themselves through routing
- Transfer vs. recompute is a threshold decision
- Result: 14% lower TTFT compared to local cache-aware scheduling alone

Evaluation Setup

Testbed

Multi-node cluster

Each node has:

8 NVIDIA-A800 80GB
GPUs

4 200Gbps NICS

KV Cache block size:
256 tokens

Each node is either prefill
or decode

Main Metrics

TBT: average of longest
10% of token arrivals

Thresholds

TBT: 100ms, 200ms,
300ms
TTFT: 30ms

GPU time during prefill

Cache hit rate

Baseline

vLLM!

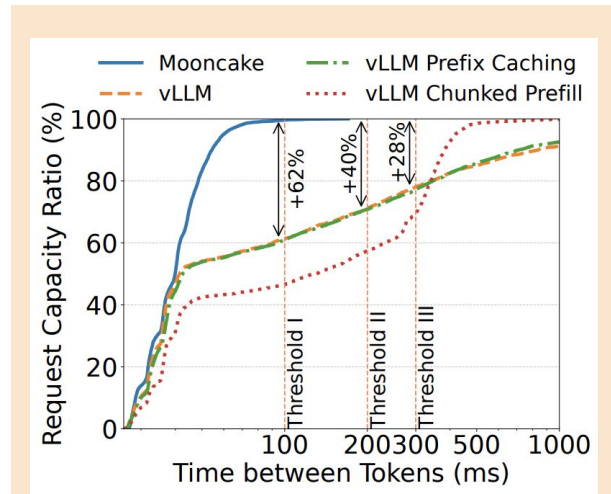
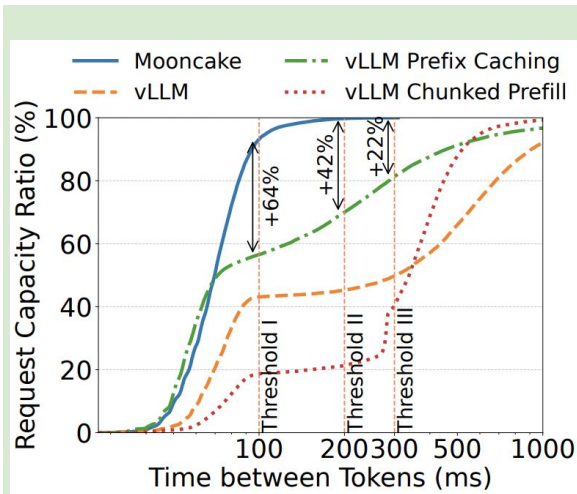
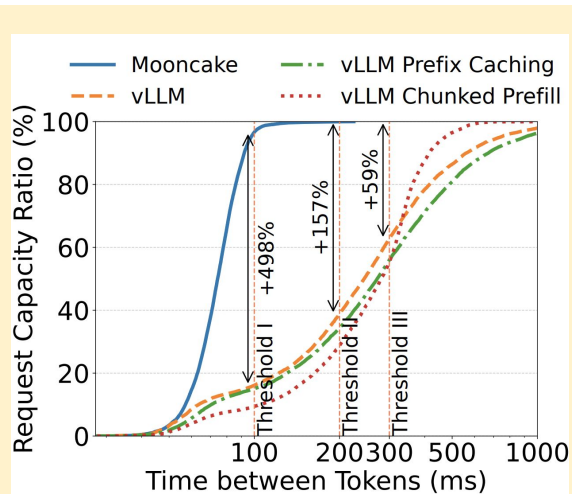
Continuous batching +
paged attention

Coupled prefill + decoding

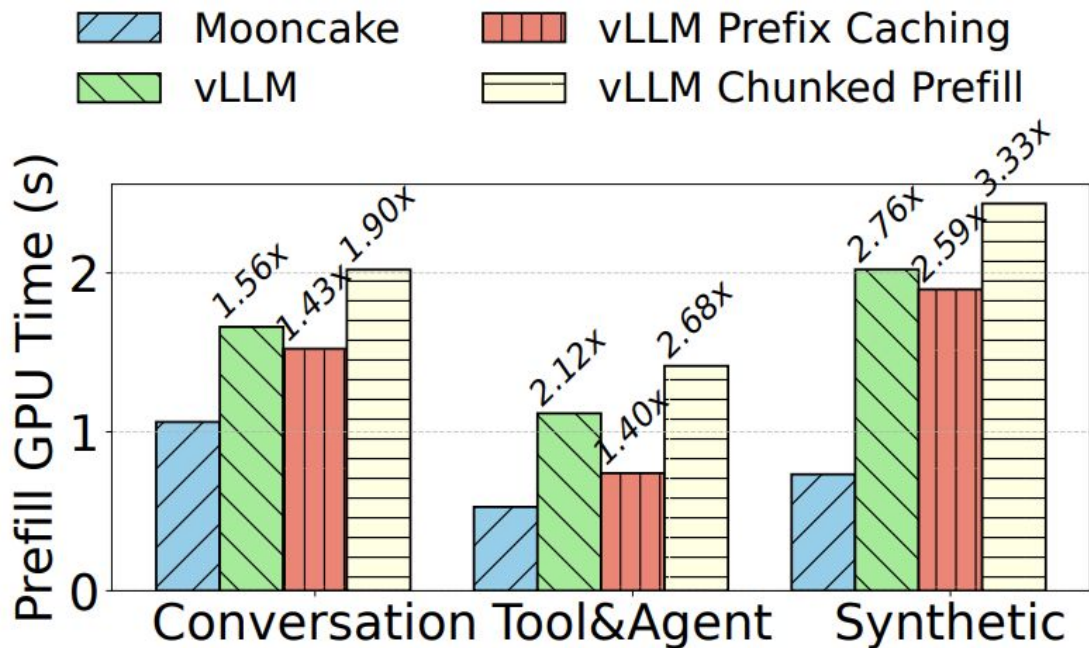
Chunked prefill + prefix
reuse

Evaluation: Request Capacity vs Workload Types

	Conversation	Tool&Agent	Synthetic
Avg Input Len	12035	8596	15325
Avg Output Len	343	182	149
Cache Ratio	40%	59%	66%
Arrival Pattern	Timestamp	Timestamp	Poisson
Num Requests	12031	23608	3993



Evaluation: GPU Time During Prefill



- **Conversation Workload:** long input lengths, low prefix cache ratio - longest GPU time
 - 36% reduction
- **Synthetic Workload:** longest average input lengths, best prefix cache ratio - intermediate GPU time
 - 64% reduction
- **Tool + Agent:** shortest average input length, good prefix cache ratio - best GPU time
 - 53% reduction

Why Global KV Cache Matters

- Local cache too small → low reuse
- Global cache → higher hit rate
- Leads to:
 - less prefill compute
 - better throughput

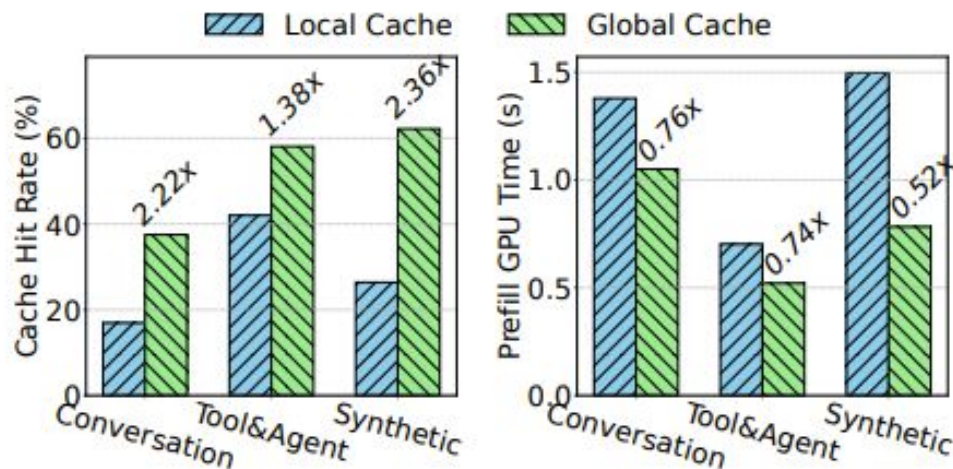


Figure 10: Cache hit rates and average GPU computation time for prefill in global and local caches.

Strengths

- High throughput (up to 5× more requests)
- Reduces GPU compute via reuse
- Works well for long-context workloads
- Global cache improves hit rate

Weaknesses

- Requires high network bandwidth
- Complex system design
- Large storage overhead
- Performance depends on prefix reuse

Future Directions

- KVCache compression (reduce storage)
- Learning-based scheduling (better than heuristics)
- Hybrid local + global caching
- Fault tolerance under node failures

Related Works

- vLLM / PagedAttention
 - Efficient KVCache management in GPU memory
 - Continuous batching for high throughput
 - Cache is local → limited reuse
- TensorRT-LLM / DeepSpeed Inference
 - Optimize GPU execution and throughput
 - Kernel-level + system optimizations
 - Focus on compute, not reuse

JITServe vs MoonCake

- **JITServe**: Poor scheduling across diverse request types (chat, API, agents)
 - Decides which requests to run and when
- **Mooncake**: Expensive prefills + limited KV cache reuse in long-context workloads
 - Decides where KV cache lives and how it's reused